

Algoritmos de Ordenación

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

8 de abril de 2026

Sorting Algorithms

Análisis de distintos algoritmos de ordenación, técnicas de solución y eficiencia.



**Universidad
Pontificia
Bolivariana**

Índice

1. Algoritmos de ordenación	2
1.1. Definición y Aplicaciones	2
1.2. Comparación de ADTs	3
1.3. Ordenación por selección	3
1.4. Ordenación por inserción	4
1.5. Ordenación Shellsort	5
1.6. Ordenación por fusión (mergesort)	7
1.7. Ordenación rápida (quicksort)	7
1.8. Resumen comparativo	10
2. Análisis por medio de árbol de llamados recursivos	11
3. Análisis por medio de recurrencias	12
4. Teorema maestro	14

1. Algoritmos de ordenación

1.1. Definición y Aplicaciones

Ordenar un conjunto de datos significa reorganizarlo en una secuencia específica, ya sea ascendente o descendente, según una relación de orden predefinida. Esta relación de orden define un conjunto totalmente ordenado, donde para cualquier par de elementos, se puede determinar cuál es mayor o menor. Por ejemplo, los números enteros tienen un orden natural: dado cualquier par de enteros, se puede determinar cuál es mayor.

Este capítulo presenta cinco algoritmos de ordenación, organizados de menor a mayor eficiencia. Primero se estudian tres algoritmos elementales — ordenación por selección, por inserción y shellsort — cuya complejidad temporal es cuadrática o subcuadrática. Luego se presentan dos algoritmos basados en la técnica de *divide y vencerás* — mergesort y quicksort — que alcanzan una complejidad de $O(n \lg n)$. Al final del capítulo se introducen herramientas de análisis para algoritmos recursivos: árboles de llamados, recurrencias y el teorema maestro.

Las aplicaciones de la ordenación son vastas:

- **Bases de datos:** La ordenación es fundamental para indexar datos y acelerar las consultas.
- **Generación de reportes:** Los datos ordenados facilitan la creación de informes claros y concisos.
- **Búsqueda:** La búsqueda de elementos en un conjunto ordenado es mucho más eficiente (e.g., búsqueda binaria).
- **Compresión de datos:** Algunos algoritmos de compresión se benefician de datos ordenados.

Para ordenar un conjunto de datos se requiere:

- **Datos:** Un conjunto de n elementos a ordenar.
- **Llave (*key*):** El campo o atributo del elemento que se usa para determinar el orden. Por ejemplo, al ordenar una lista de estudiantes, la llave podría ser el nombre, la cédula o la calificación.
- **Relación de orden:** Define cómo se comparan las llaves entre sí ($<$, $\leq$, etc.).

A lo largo del capítulo, se utilizan las siguientes propiedades para caracterizar los algoritmos de ordenación:

- **Estable:** Un algoritmo de ordenación es estable si preserva el orden relativo de los elementos con claves iguales. Es decir, si dos elementos tienen la misma clave, aparecerán en el mismo orden en la salida que en la entrada. Por ejemplo, si una lista de estudiantes ordenada por nombre se reordena por calificación con un algoritmo estable, los estudiantes con la misma calificación permanecerán en orden alfabético.
- **Adaptativo:** Un algoritmo es adaptativo si su complejidad temporal mejora cuando la entrada está parcialmente ordenada. Por ejemplo, la ordenación por inserción es $O(n)$ cuando la entrada ya está ordenada.
- **In-situ (*in-place*):** Un algoritmo es in-situ si utiliza una cantidad constante de memoria adicional, es decir, $O(1)$ espacio auxiliar (sin contar la pila de recursión en algoritmos recursivos).

Estas propiedades se ilustrarán con ejemplos concretos a lo largo del capítulo.

1.2. Comparación de ADTs

Para comparar ADT, los lenguajes de programación ofrecen mecanismos específicos:

- **Java:** La interfaz `Comparable` permite definir un orden natural para los objetos de una clase. La clase debe implementar el método `compareTo()`, que devuelve un valor negativo, cero o positivo si el objeto es menor, igual o mayor que el objeto con el que se compara.
[Interface Comparable<T>](#)
[Interface Comparator<T>](#)
- **C#:** La interfaz `IComparable` cumple una función similar a `Comparable` en Java. Las clases que implementan `IComparable` deben proporcionar una implementación del método `CompareTo()`.
[IComparable<T> Interface](#)
[IComparer<T> Interface](#)
- **Python:** La comparación de objetos se basa en métodos especiales como `__lt__` (menor que), `__le__` (menor o igual que), `__eq__` (igual a), `__gt__` (mayor que) y `__ge__` (mayor o igual que). Al implementar estos métodos, se define cómo se comparan los objetos de una clase. Por ejemplo, implementar `__le__` permite utilizar el operador `<=` para comparar instancias de la clase. En la práctica, no es necesario implementar todos los métodos: el decorador `@functools.total_ordering` genera automáticamente los métodos de comparación faltantes a partir de `__eq__` y uno de los métodos de ordenamiento (e.g., `__lt__`).
[Rich comparison methods: __lt__, __le__, __gt__, __ge__](#).
[functools.total_ordering](#)
[Unravelling rich comparison operators](#)

Una vez definido el mecanismo de comparación entre objetos, se pueden estudiar los algoritmos que utilizan estas comparaciones para reorganizar los elementos en un orden determinado. A continuación se presentan los algoritmos de ordenación, comenzando con los más simples.

1.3. Ordenación por selección

La ordenación por selección funciona encontrando el elemento mínimo en la parte no ordenada del arreglo y luego intercambiándolo con el elemento en la posición actual. En cada iteración, el arreglo se divide conceptualmente en dos partes: una parte ordenada (a la izquierda) y una parte no ordenada (a la derecha). El algoritmo selecciona el elemento más pequeño de la parte no ordenada y lo coloca al final de la parte ordenada. Ver algoritmo 1.

Ejemplo Traza de la ordenación por selección sobre el arreglo [29, 10, 14, 37, 13]:

Iteración	Arreglo	Acción
—	[29, 10, 14, 37, 13]	Estado inicial
$i = 0$	[10 , 29, 14, 37, 13]	Mín. es 10 (pos. 1), intercambio con pos. 0
$i = 1$	[10, 13 , 14, 37, 29]	Mín. es 13 (pos. 4), intercambio con pos. 1
$i = 2$	[10, 13, 14 , 37, 29]	Mín. es 14 (pos. 2), ya está en su lugar
$i = 3$	[10, 13, 14, 29 , 37]	Mín. es 29 (pos. 4), intercambio con pos. 3

Resultado: [10, 13, 14, 29, 37].

Análisis de la eficiencia de la ordenación por selección:

Algorithm 1 Ordenación por selección**Require:** $T[0 \dots n - 1]$ es un vector de objetos comparables**Ensure:** $T[0 \dots n - 1]$ el vector de entrada ordenado ascendentemente

```

1: function SELECTIONSORT( $T[0 \dots n - 1]$ )
2:   for  $i \leftarrow 0$  to  $n - 2$  do
3:      $minj \leftarrow i$ 
4:      $minx \leftarrow T[i]$ 
5:     for  $j \leftarrow i + 1$  to  $n - 1$  do
6:       if  $T[j] < minx$  then
7:          $minj \leftarrow j$ 
8:          $minx \leftarrow T[j]$ 
9:       end if
10:    end for
11:     $T[minj] \leftarrow T[i]$ 
12:     $T[i] \leftarrow minx$ 
13:  end for
14: end function

```

La ordenación por selección tiene una complejidad temporal de $O(n^2)$ en todos los casos (peor, promedio y mejor). Esto se debe a que siempre realiza dos bucles anidados para encontrar el elemento mínimo y colocarlo en su posición correcta.

Características de la ordenación por selección:

- **Cuadrático en comparaciones:** Realiza un número de comparaciones proporcional a n^2 .
- **Lineal en intercambios:** Realiza exactamente $n - 1$ intercambios (uno por cada iteración del bucle externo), es decir, $O(n)$ intercambios. Esto la hace útil cuando los intercambios son costosos.
- **No adaptativo:** Su rendimiento no se ve afectado por el orden inicial de los datos.
- **In-situ:** No requiere memoria adicional.
- **No estable:** No preserva el orden relativo de los elementos con claves iguales. Por ejemplo, al ordenar $[2_a, 2_b, 1]$, el intercambio de 2_a con 1 produce $[1, 2_b, 2_a]$, invirtiendo el orden relativo de los dos elementos con clave 2.

1.4. Ordenación por inserción

La ordenación por inserción funciona construyendo incrementalmente un subarreglo ordenado desde el principio del arreglo. En cada iteración, toma el siguiente elemento y lo inserta en la posición correcta dentro del subarreglo ya ordenado, desplazando los elementos mayores hacia la derecha. El proceso es análogo a cómo se ordenan las cartas en la mano: se toma una carta nueva y se desliza hacia la izquierda hasta encontrar su lugar. Ver algoritmo 2.

Ejemplo Traza de la ordenación por inserción sobre el arreglo $[29, 10, 14, 37, 13]$:

Iteración	Elemento	Arreglo
—	—	[29, 10, 14, 37, 13]
$i = 1$	10	[10 , 29, 14, 37, 13]
$i = 2$	14	[10, 14 , 29, 37, 13]
$i = 3$	37	[10, 14, 29, 37 , 13]
$i = 4$	13	[10, 13 , 14, 29, 37]

En la iteración $i = 4$, el elemento 13 se desplaza tres posiciones a la izquierda hasta quedar entre 10 y 14. Resultado: [10, 13, 14, 29, 37].

Algorithm 2 Ordenación por inserción

Require: $T[0 \dots n - 1]$ es un vector de objetos comparables

Ensure: $T[0 \dots n - 1]$ el vector de entrada ordenado ascendentemente

```

1: function INSERTIONSORT( $T[0 \dots n - 1]$ )
2:   for  $i \leftarrow 1$  to  $n - 1$  do
3:      $x \leftarrow T[i]$ 
4:      $j \leftarrow i - 1$ 
5:     while  $j \geq 0$  and  $x < T[j]$  do
6:        $T[j + 1] \leftarrow T[j]$            ▷ Los elementos mayores a  $x$  se desplazan a la derecha
7:        $j \leftarrow j - 1$ 
8:     end while
9:      $T[j + 1] \leftarrow x$ 
10:  end for
11: end function

```

Análisis de la eficiencia de la ordenación por inserción:

- **Peor caso:** $O(n^2)$. Ocurre cuando el arreglo está ordenado en orden inverso.
- **Mejor caso:** $O(n)$. Ocurre cuando el arreglo ya está ordenado. En este caso, solo realiza una comparación en cada iteración del bucle principal.
- **Caso promedio:** $O(n^2)$.

Características de la ordenación por inserción:

- **Adaptativo:** Su rendimiento mejora si el arreglo está parcialmente ordenado.
- **Estable:** Preserva el orden relativo de los elementos con claves iguales.
- **In-situ:** No requiere memoria adicional.
- **Simple de implementar:** Es un algoritmo relativamente fácil de entender e implementar.

1.5. Ordenación Shellsort

La ordenación por inserción mueve cada elemento solo una posición a la vez, lo que resulta costoso cuando un elemento pequeño está lejos de su posición final. Shellsort aborda esta limitación permitiendo mover elementos a posiciones lejanas.

La idea central es un parámetro h llamado *incremento* o *salto*. El algoritmo primero ordena elementos que están separados por h posiciones — se dice que el arreglo queda h -ordenado. Por ejemplo, un arreglo está 3-ordenado si $T[0] \leq T[3] \leq T[6]$, $T[1] \leq T[4] \leq T[7]$, etc. Luego reduce el valor de h y repite el proceso.

Cuando h llega a 1, el algoritmo se convierte en una ordenación por inserción convencional. Sin embargo, para ese momento el arreglo ya está parcialmente ordenado gracias a las pasadas anteriores, lo que hace que la inserción final sea mucho más eficiente. La secuencia de incrementos utilizada en el algoritmo 3 es la de Knuth: $h_0 = 1$, $h_1 = 3(1) + 1 = 4$, $h_2 = 3(4) + 1 = 13$, $h_3 = 3(13) + 1 = 40$, ..., definida por la recurrencia $h_{k+1} = 3h_k + 1$. Ver algoritmo 3.

Algorithm 3 Ordenación shellsort

Require: $T[0 \dots n-1]$ es un vector de objetos comparables

Ensure: $T[0 \dots n-1]$ el vector de entrada ordenado ascendentemente

```

1: function SHELLSORT( $T[0 \dots n-1]$ )
2:    $h \leftarrow 1$ 
3:   while  $h < n/3$  do
4:      $h \leftarrow 3h + 1$ 
5:   end while
6:   while  $h \geq 1$  do
7:     for  $i \leftarrow h$  to  $n-1$  do
8:        $x \leftarrow T[i]$ 
9:        $j \leftarrow i - h$ 
10:      while  $j \geq 0$  and  $x < T[j]$  do
11:         $T[j+h] \leftarrow T[j]$       ▷ Los elementos mayores a  $x$  se desplazan a la derecha
12:         $j \leftarrow j - h$ 
13:      end while
14:       $T[j+h] \leftarrow x$ 
15:    end for
16:     $h \leftarrow \lfloor h/3 \rfloor$ 
17:  end while
18: end function

```

Análisis de la eficiencia de la ordenación Shellsort:

La complejidad temporal de Shellsort no tiene una fórmula cerrada general: depende de la secuencia de incrementos utilizada, y su análisis exacto permanece como un problema abierto en informática teórica.

- **Complejidad empírica:** En la práctica, Shellsort muestra un rendimiento significativamente mejor que la ordenación por selección e inserción, especialmente para arreglos de tamaño mediano. Con la secuencia de incrementos original de Shell ($n/2, n/4, \dots, 1$), la complejidad es $O(n^2)$. Con la secuencia de incrementos utilizada en el algoritmo 3 ($1, 4, 13, 40, 121, \dots$, definida por $h = 3h + 1$), la complejidad es $O(n^{3/2})$.

Características de la ordenación Shellsort:

- **No estable:** No preserva el orden relativo de los elementos con claves iguales.
- **In-situ:** No requiere memoria adicional.

- **Parcialmente adaptativo:** Su rendimiento mejora en la práctica cuando la entrada está parcialmente ordenada, ya que las pasadas con incrementos decrecientes aprovechan el orden parcial generado por pasadas anteriores. Sin embargo, la magnitud de esta mejora depende de la secuencia de incrementos utilizada.

1.6. Ordenación por fusión (mergesort)

Los algoritmos anteriores tienen complejidad cuadrática o cercana a ella. A continuación se presentan dos algoritmos que logran una complejidad de $O(n \lg n)$ utilizando la técnica de *divide y vencerás*.

Mergesort es un algoritmo de ordenación que sigue tres pasos:

1. **Dividir:** Se divide el arreglo en dos mitades.
2. **Conquistar:** Se ordena cada mitad recursivamente.
3. **Combinar:** Se fusionan (*merge*) las dos mitades ordenadas en un solo arreglo ordenado.

La recursión se detiene cuando cada subarreglo contiene un solo elemento (que se considera ordenado por definición). Ver algoritmo 4.

La operación clave es la función MERGE, que combina dos subarreglos ya ordenados en uno solo. La función recorre ambas mitades simultáneamente usando dos índices (i y j): en cada paso, compara los elementos frontales de cada mitad y copia el menor al arreglo de salida. Cuando una de las mitades se agota, los elementos restantes de la otra se copian directamente.

Ejemplo Fusión de las mitades ordenadas $[10, 14, 29]$ y $[13, 37]$: se comparan los elementos frontales: $10 < 13$, se copia 10. Luego $14 > 13$, se copia 13. Después $14 < 37$, se copia 14. Después $29 < 37$, se copia 29. Finalmente, se copia el elemento restante 37. Resultado: $[10, 13, 14, 29, 37]$.

Análisis de la eficiencia de la ordenación por fusión:

La ordenación por fusión tiene una complejidad temporal de $O(n \lg n)$ en todos los casos (peor, promedio y mejor), dado que divide el arreglo en mitades ($\lg n$ niveles de recursión) y en cada nivel la fusión procesa todos los n elementos. Esta complejidad se demuestra formalmente en la sección 2.

Características de la ordenación por fusión:

- **Complejidad temporal garantizada:** $O(n \lg n)$ en todos los casos, a diferencia de quicksort que puede degradarse a $O(n^2)$.
- **Complejidad espacial:** $O(n)$. Requiere memoria adicional para el arreglo auxiliar utilizado en la fusión.
- **Estable:** Preserva el orden relativo de los elementos con claves iguales.
- **No adaptativo:** Su rendimiento no se ve afectado por el orden inicial de los datos.

1.7. Ordenación rápida (quicksort)

Quicksort es un algoritmo de ordenación muy eficiente que también utiliza la técnica de *divide y vencerás*. Selecciona un elemento del arreglo como “pivote” y luego particiona el arreglo en dos subarreglos: uno con elementos menores que el pivote y otro con elementos mayores. El pivote

Algorithm 4 Ordenación por fusión (Mergesort)**Require:** $T[0 \dots n - 1]$ es un vector de objetos comparables.**Ensure:** $T[0 \dots n - 1]$ el vector de entrada ordenado ascendentemente.

```

1: function MERGE( $T, aux, lo, mid, hi$ )
2:   for  $k \leftarrow lo$  to  $hi$  do
3:      $aux[k] \leftarrow T[k]$ 
4:   end for
5:    $i \leftarrow lo$ 
6:    $j \leftarrow mid + 1$ 
7:   for  $k \leftarrow lo$  to  $hi$  do
8:     if  $i > mid$  then
9:        $T[k] \leftarrow aux[j]$ 
10:       $j \leftarrow j + 1$ 
11:    else if  $j > hi$  then
12:       $T[k] \leftarrow aux[i]$ 
13:       $i \leftarrow i + 1$ 
14:    else if  $aux[j] < aux[i]$  then
15:       $T[k] \leftarrow aux[j]$ 
16:       $j \leftarrow j + 1$ 
17:    else
18:       $T[k] \leftarrow aux[i]$ 
19:       $i \leftarrow i + 1$ 
20:    end if
21:  end for
22: end function

```

```

23: function MSORT( $T, aux, lo, hi$ )
24:   if  $hi \leq lo$  then
25:     return
26:   end if
27:    $mid \leftarrow lo + \lfloor (hi - lo)/2 \rfloor$ 
28:   MSORT( $T, aux, lo, mid$ )
29:   MSORT( $T, aux, mid + 1, hi$ )
30:   MERGE( $T, aux, lo, mid, hi$ )
31: end function

```

```

32: function MERGESORT( $T[0 \dots n - 1]$ )
33:    $n \leftarrow$  longitud de  $T$ 
34:    $aux \leftarrow$  un nuevo vector de tamaño  $n$ 
35:   MSORT( $T, aux, 0, n - 1$ )
36: end function

```

queda en su posición final ordenada. Los subarreglos se ordenan recursivamente. La eficiencia de Quicksort depende en gran medida de la elección del pivote. Ver el algoritmo 5.

La función PARTITION implementa la partición usando dos índices que recorren el arreglo en direcciones opuestas: i avanza desde la izquierda buscando un elemento $\geq$ al pivote, y j retrocede desde la derecha buscando un elemento $\leq$ al pivote. Cuando ambos encuentran elementos “fuera de lugar”, los intercambia. El proceso termina cuando los índices se cruzan, momento en el cual el pivote se coloca en la posición de cruce.

Algorithm 5 Ordenación rápida (Quicksort)

Require: $T[0 \dots n - 1]$ es un vector de objetos comparables.

Ensure: $T[0 \dots n - 1]$ el vector de entrada ordenado ascendentemente.

```

1: function PARTITION( $T, lo, hi$ )
2:    $i \leftarrow lo$ 
3:    $j \leftarrow hi + 1$ 
4:    $v \leftarrow T[lo]$  ▷ Elemento pivote
5:   while  $True$  do
6:     repeat
7:        $i \leftarrow i + 1$ 
8:     until  $T[i] \geq v$  or  $i = hi$ 
9:     repeat
10:       $j \leftarrow j - 1$ 
11:    until  $T[j] \leq v$  or  $j = lo$ 
12:    if  $i \geq j$  then
13:      break
14:    end if
15:    Intercambiar  $T[i] \leftrightarrow T[j]$ 
16:  end while
17:  Intercambiar  $T[lo] \leftrightarrow T[j]$ 
18:  return  $j$ 
19: end function

20: function QSORT( $T, lo, hi$ )
21:   if  $hi \leq lo$  then
22:     return
23:   end if
24:    $j \leftarrow$  PARTITION( $T, lo, hi$ )
25:   QSORT( $T, lo, j - 1$ )
26:   QSORT( $T, j + 1, hi$ )
27: end function

28: function QUICKSORT( $T[0 \dots n - 1]$ )
29:    $n \leftarrow$  longitud de  $T$ 
30:   QSORT( $T, 0, n - 1$ )
31: end function

```

Análisis de la eficiencia de la ordenación rápida:

La eficiencia de Quicksort depende en gran medida de la elección del pivote. El análisis formal

se presenta en la sección 2.

- **Peor caso:** $O(n^2)$. Ocurre cuando el pivote es siempre el elemento más pequeño o el más grande del arreglo (por ejemplo, cuando el arreglo ya está ordenado y se elige el primer elemento como pivote). En este caso, la partición divide el arreglo en un subarreglo de tamaño 0 y otro de tamaño $n - 1$, lo que lleva a una recursión de profundidad n . Para evitar este caso, se puede aleatorizar la elección del pivote.
- **Mejor caso:** $O(n \lg n)$. Ocurre cuando el pivote divide el arreglo en dos subarreglos de tamaño aproximadamente igual. En este caso, la recursión tiene una profundidad de $\lg n$.
- **Caso promedio:** $O(n \lg n)$. Este resultado asume que la entrada es una permutación aleatoria uniforme (o que el pivote se elige aleatoriamente). Con el pivote determinista en $T[lo]$, el peor caso ocurre con entradas ya ordenadas.

Características de la ordenación rápida:

- **In-situ:** No requiere memoria adicional aparte de la pila de recursión, que es $O(\lg n)$ en el caso promedio y $O(n)$ en el peor caso.
- **No estable:** No preserva el orden relativo de los elementos con claves iguales.
- **No adaptativo:** Su rendimiento no mejora con entradas parcialmente ordenadas. De hecho, con el pivote determinista en $T[lo]$, una entrada ya ordenada produce el peor caso $O(n^2)$.
- **Muy eficiente en la práctica:** A pesar de su complejidad en el peor caso, Quicksort es uno de los algoritmos de ordenación más rápidos en la práctica, especialmente para arreglos grandes.

1.8. Resumen comparativo

La tabla 1 resume las principales características de los cinco algoritmos de ordenación estudiados en este capítulo.

Cuadro 1: Comparación de algoritmos de ordenación

Algoritmo	Mejor	Promedio	Peor	Estable	Adaptativo	In-situ
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	No	Sí
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	Sí	Sí	Sí
Shellsort	—	—	$O(n^{3/2})$	No	Parcial	Sí
Mergesort	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	Sí	No	No
Quicksort	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$	No	No	Sí

Observaciones:

- La complejidad de Shellsort depende de la secuencia de incrementos; los valores mostrados corresponden a la secuencia de Knuth ($h = 3h + 1$).
- Quicksort es in-situ pero requiere espacio $O(\lg n)$ para la pila de recursión ($O(n)$ en el peor caso).
- Mergesort requiere $O(n)$ de espacio adicional para el arreglo auxiliar.

En las secciones anteriores se presentaron las complejidades temporales de mergesort y quicksort sin demostrarlas formalmente. Para justificar esos resultados, se necesitan herramientas de análisis para algoritmos recursivos. Las técnicas de esta sección se aplican a algoritmos recursivos; los algoritmos elementales (selección, inserción, shellsort) se analizaron directamente mediante conteo de operaciones en las secciones anteriores. A continuación se presentan tres técnicas: el árbol de llamados recursivos, el método de sustitución de recurrencias y el teorema maestro.

2. Análisis por medio de árbol de llamados recursivos

El análisis por medio de árbol de llamados recursivos es una técnica que permite visualizar y analizar el costo de un algoritmo recursivo. Se construye un árbol donde cada nodo representa una llamada a la función recursiva. La raíz del árbol representa la llamada inicial, y los hijos de cada nodo representan las llamadas recursivas realizadas por ese nodo.

Para analizar el costo del algoritmo, se contabiliza la operación seleccionada como modelo de costo para el algoritmo en cada nodo del árbol. El costo total del algoritmo es la suma de los costos de todos los nodos del árbol.

Una forma común de usar el árbol es sumar el costo de todos los nodos en el mismo nivel, y luego sumar los costos de todos los niveles. Para un algoritmo como **mergesort**, el costo en cada nivel del árbol de recursión (exceptuando las hojas) es proporcional a n , ya que la operación **merge** procesa todos los elementos en ese nivel. Dado que el árbol tiene una altura de $\lg(n)$, el costo total es la suma del costo de cada nivel, lo que resulta en un costo total de $\Theta(n \lg n)$.

Ejemplo Árbol de llamados recursivos para el algoritmo mergesort

Consideremos el algoritmo mergesort para ordenar una lista de n elementos. El árbol de llamadas recursivas tendrá una estructura donde cada nivel representa una división del arreglo en dos mitades. En cada nodo, los modelos de costo dominante son la comparación de elementos y los accesos al arreglo en los llamados a la operación **merge**. En la figura 1 se ilustra el cálculo del número de comparaciones de **mergesort** usando el árbol de llamados recursivos.

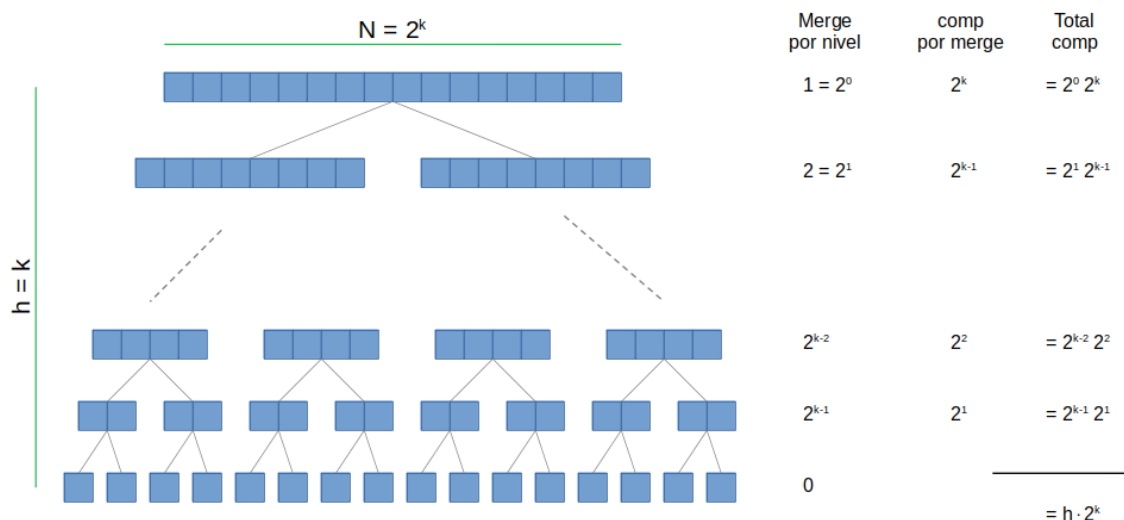


Figura 1: Árbol de invocaciones recursivas para mergesort

Limitaciones de esta metodología: Esta metodología puede ser compleja de aplicar si el árbol de llamadas no sigue un patrón regular, o si el costo de las operaciones en cada nodo varía significativamente. La figura 2 ilustra una posible secuencia de llamadas recursivas para **quicksort**, mostrando las irregularidades del árbol: los subárboles no tienen la misma altura (el árbol no está balanceado) y el número de nodos por nivel varía, lo que dificulta sumar los costos por nivel.

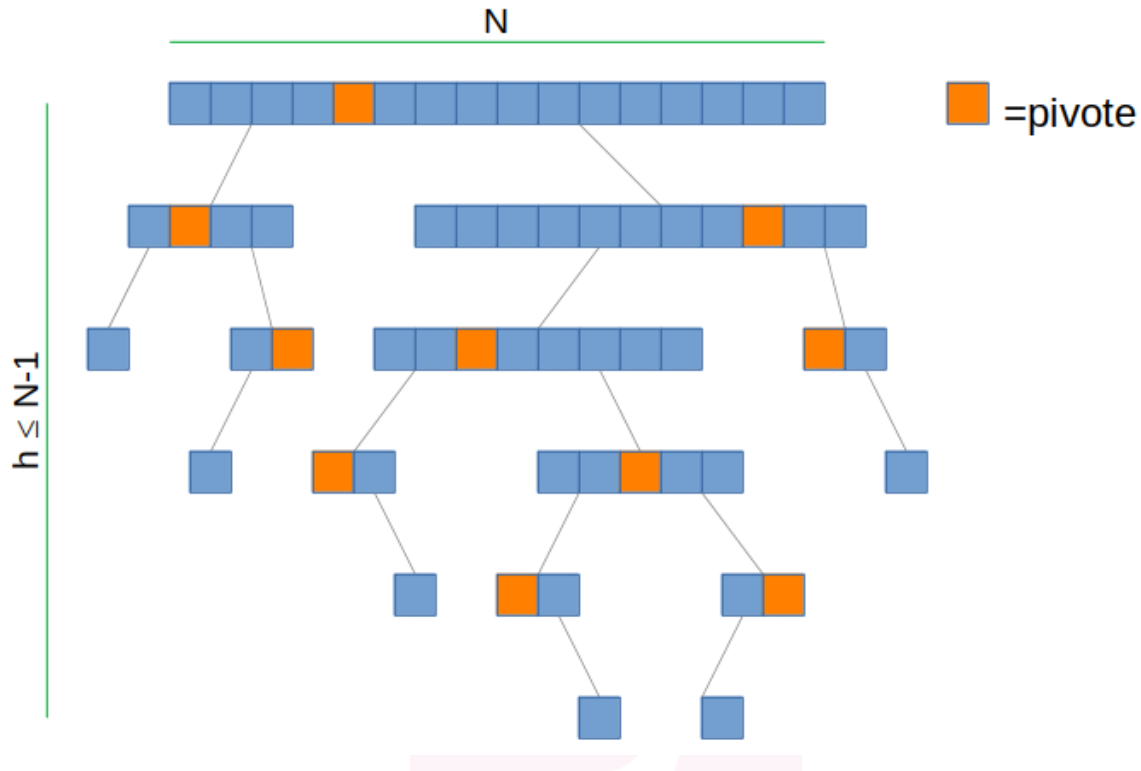


Figura 2: Árbol de invocaciones recursivas para **quicksort**

3. Análisis por medio de recurrencias

El análisis por medio de recurrencias es una técnica que permite describir el costo de un algoritmo recursivo en función del tamaño de la entrada. Una recurrencia es una ecuación que define una función en términos de sí misma, con uno o más casos base.

La recurrencia expresa el costo de una invocación del algoritmo como la suma del trabajo realizado en esa invocación más el costo de los llamados recursivos.

Método de sustitución Este método permite “resolver” la recurrencia, es decir, encontrar una fórmula cerrada (no recursiva) para la misma función. El método de sustitución, también conocido como *expansión* o *desenrollado* (unrolling), implica expandir la recurrencia repetidamente sustituyendo los términos recursivos hasta llegar al caso base. Luego se simplifica la expresión resultante para obtener una fórmula explícita.

Ejemplo Análisis por medio de recurrencias para el algoritmo **mergesort**

Sea $C(N)$ el número de comparaciones que realiza **mergesort** para ordenar un arreglo de tamaño N (ver algoritmo 4). Al invocar el algoritmo, el arreglo se subdivide en dos mitades de tamaños

$\lceil N/2 \rceil$ y $\lfloor N/2 \rfloor$, que requieren $C(\lceil N/2 \rceil)$ y $C(\lfloor N/2 \rfloor)$ comparaciones respectivamente. Además, la operación **merge** realiza como máximo $N - 1$ comparaciones. Por lo tanto, se obtiene la siguiente recurrencia:

$$C(N) = N + C\left(\left\lceil \frac{N}{2} \right\rceil\right) + C\left(\left\lfloor \frac{N}{2} \right\rfloor\right)$$

Para simplificar el análisis, asumimos que N es una potencia de 2, es decir, $N = 2^k$ para algún entero k (el resultado $\Theta(N \lg N)$ es válido para todo N , ya que la función $N \lg N$ es monótonamente creciente). La recurrencia se convierte en: $C(N) = 2C(N/2) + N$, con $C(1) = 0$

Ahora, aplicamos el método de sustitución (también conocido como expansión o "unrolling") para resolver la recurrencia:

$$\begin{aligned} C(N) &= 2C(N/2) + N \\ &= 2(2C(N/4) + N/2) + N && \text{Sustituyendo } C(N/2) \\ &= 4C(N/4) + N + N = 4C(N/4) + 2N \\ &= 4(2C(N/8) + N/4) + 2N && \text{Sustituyendo } C(N/4) \\ &= 8C(N/8) + N + 2N = 8C(N/8) + 3N \\ &\vdots \\ &= 2^k C(N/2^k) + kN && \text{Después de } k \text{ sustituciones} \end{aligned}$$

Sabemos que $N = 2^k$, lo que implica que $k = \lg N$. También, $N/2^k = 1$. Sustituyendo estos valores en la ecuación:

$$C(N) = 2^{\lg N} C(1) + (\lg N)N$$

Dado que $2^{\lg N} = N$ y el caso base es $C(1) = 0$:

$$C(N) = N \cdot 0 + N \lg N = N \lg N$$

Por lo tanto, la complejidad de **mergesort** es $\Theta(N \lg N)$.

Ejemplo Análisis por medio de recurrencias para el algoritmo **quicksort**

El análisis de Quicksort (algoritmo 5) por medio de recurrencias es más complejo debido a que el tamaño de los subproblemas depende de la elección del pivote. En el mejor de los casos, el pivote divide el arreglo en dos subarreglos de tamaño aproximadamente igual. En el peor de los casos, el pivote resulta ser el elemento más pequeño o el más grande, lo que lleva a un subarreglo de tamaño 0 y otro de tamaño $n - 1$.

Peor caso En el peor caso, la recurrencia para el número de comparaciones $C(n)$ es:

$$C(n) = C(n - 1) + n$$

Esto se debe a que, en cada paso, se realiza una partición que requiere aproximadamente n comparaciones y se genera un subproblema de tamaño $n - 1$. Desarrollando la recurrencia:

$$\begin{aligned}
C(n) &= C(n-1) + n \\
&= C(n-2) + (n-1) + n \\
&= C(n-3) + (n-2) + (n-1) + n \\
&\vdots \\
&= C(1) + 2 + 3 + \dots + n
\end{aligned}$$

Dado que $C(1) = 0$, la suma resultante es la suma de los enteros desde 2 hasta n , que equivale a la suma de los primeros n enteros menos 1:

$$C(n) = \sum_{i=2}^n i = \frac{n(n+1)}{2} - 1 = \Theta(n^2)$$

Mejor caso En el mejor caso, cada partición divide el arreglo en dos subarreglos de tamaño aproximadamente $n/2$. La recurrencia para el número de comparaciones $C(n)$ es:

$$C(n) = 2C(n/2) + n$$

Esta recurrencia es similar a la de Mergesort. Aplicando el método de sustitución (de manera análoga al análisis de Mergesort), se obtiene que la solución es $C(n) = \Theta(n \lg n)$. Alternativamente, se puede aplicar el teorema maestro (ver sección 4), con $a = 2$, $b = 2$, y $f(n) = n$, llegando a la misma conclusión.

Caso promedio El análisis del caso promedio es más intrincado, pero se puede demostrar que también tiene un costo de $\Theta(n \lg n)$. La recurrencia para el caso promedio es:

$$C(n) = n + \frac{1}{n} \sum_{i=1}^n (C(i-1) + C(n-i))$$

Esta recurrencia refleja que cada posible pivote tiene la misma probabilidad de ser elegido (lo cual se cumple cuando la entrada es una permutación aleatoria uniforme, o cuando el pivote se elige aleatoriamente), y se suman los costos de las dos subllamadas recursivas resultantes. La demostración de que $C(n) = \Theta(n \lg n)$ requiere manipulaciones algebraicas más avanzadas y queda fuera del alcance de este capítulo. El lector interesado puede consultar las referencias bibliográficas para una derivación completa.

4. Teorema maestro

El teorema maestro proporciona una solución directa para recurrencias de la forma $T(n) = aT(n/b) + f(n)$, donde $a \geq 1$ y $b > 1$ son constantes, y $f(n)$ es una función asintóticamente positiva. Este teorema es útil para determinar el orden de crecimiento de algoritmos recursivos.

Expresión para el teorema maestro y distintos casos de aplicación:

El teorema maestro establece que, dependiendo de la relación entre $f(n)$ y $n^{\log_b a}$, se pueden determinar tres casos:

- Caso 1: Si $f(n) = O(n^{\log_b a - \epsilon})$ para alguna constante $\epsilon > 0$, entonces $T(n) = \Theta(n^{\log_b a})$.

- Caso 2: Si $f(n) = \Theta(n^{\log_b a})$, entonces $T(n) = \Theta(n^{\log_b a} \log n)$.
- Caso 3: Si $f(n) = \Omega(n^{\log_b a + \epsilon})$ para alguna constante $\epsilon > 0$, y si $af(n/b) \leq cf(n)$ para alguna constante $c < 1$ y n suficientemente grande, entonces $T(n) = \Theta(f(n))$.

Ejemplo Aplicación del teorema maestro para el algoritmo **mergesort**

Para el algoritmo **mergesort**, la recurrencia es $C(n) = 2C(n/2) + O(n)$. Aquí, $a = 2$, $b = 2$, y $f(n) = O(n)$. Por lo tanto, $n^{\log_b a} = n^{\log_2 2} = n$. Como $f(n) = \Theta(n)$, estamos en el Caso 2 del teorema maestro, y la solución es $C(n) = \Theta(n \lg n)$.

Limitaciones del teorema maestro: El teorema maestro no se puede aplicar a todas las recurrencias. Solo funciona para recurrencias de la forma $T(n) = aT(n/b) + f(n)$ donde $a \geq 1$, $b > 1$, y $f(n)$ debe cumplir las condiciones específicas de alguno de los tres casos. Por ejemplo, no se puede aplicar directamente a recurrencias como $T(n) = T(n-1) + n$ (quicksort peor caso) o $T(n) = T(n/2) + T(n/4) + n$ (donde los subproblemas tienen tamaños diferentes).